DATABASE MANAGEMENT SYSTEMS
MODULE 4

SQL joins, subqueries, views, TCL, DCL and PL/SQL

Exam Focus. Study strategy. Learn each definition with its syntax, then practise tracing a query. For PL/SQL, memorize the block structure and the distinction between implicit and explicit cursors. Examples follow the supplied slides' Oracle-style syntax.

Contents

1 SQL Joins **2**
- 1.1 Meaning and parts of a join . . . 2
- 1.2 Join forms . . . 2
- 1.3 Syntax and examples . . . 2

2 Subqueries **3**
- 2.1 Definition and execution . . . 3
- 2.2 Types by result cardinality . . . 3
- 2.3 Placement and examples . . . 3
- 2.4 Correlated and uncorrelated subqueries . . . 4

3 Views **4**
- 3.1 Definition, purpose and creation . . . 4
- 3.2 Simple and complex views . . . 4
- 3.3 View updates . . . 4

4 Transaction Control Language (TCL) **5**

5 Data Control Language (DCL) **5**

6 PL/SQL **5**
- 6.1 What PL/SQL adds . . . 5
- 6.2 Block structure . . . 5
- 6.3 Variables, assignment and output . . . 6
- 6.4 Decision and repetition . . . 6
- 6.5 Implicit cursor and SQL% attributes . . . 6
- 6.6 Explicit cursors . . . 7

7 Important Exam Questions **7**

8 Last-Minute Revision **8**

SQL Joins

Meaning and parts of a join

A join combines rows from two relations using a condition that determines which rows match. Its result is another relation. A **join condition** specifies matching rows and output attributes. A **join type** determines what happens to unmatched rows.

> **Exam Focus. Likely question: Define a join.** A join is a relational operation that combines tuples from two tables according to a specified condition and returns a relation containing the matching (and, for outer joins, selected unmatched) tuples.

Join forms

- **Inner join:** returns only rows that satisfy the join condition.
- **Left outer join:** returns every left-table row and matching right-table data; unmatched right-side columns are NULL.
- **Right outer join:** returns every right-table row and matching left-table data; unmatched left-side columns are NULL.
- **Full outer join:** returns matched rows plus unmatched rows from both sides, padding missing columns with NULL.
- **Cross join:** pairs every row of one table with every row of the other; with m and n rows, the result has $m \times n$ rows.
- **Self join:** joins a table to itself using aliases, for example to match employees to managers.

Syntax and examples

```
-- Explicit inner join
SELECT e.empname, d.deptname
FROM employee e INNER JOIN department d
  ON e.deptno = d.deptno;

-- Left outer join: preserve every department
SELECT d.deptno, d.deptname, e.empname
FROM department d LEFT OUTER JOIN employee e
  ON d.deptno = e.deptno;

-- Traditional join style shown in the slides
SELECT e.empname, d.deptname
FROM employee e, department d
WHERE e.deptno = d.deptno;
```

The comma-separated style forms candidate pairs and the WHERE condition filters them. If the join condition is omitted, the result can become a Cartesian product. Use table aliases and qualify columns that share names.

Type	Rows preserved	Unmatched-side values
Inner	Only matching rows	Excluded
Left outer	All left rows	Right columns are NULL
Right outer	All right rows	Left columns are NULL
Full outer	All rows from both sides	Missing side is NULL

> **Do Not Confuse.** A condition placed in WHERE can remove NULL-extended rows from an outer join, effectively changing its result. Put matching criteria in ON when unmatched rows must remain.

Subqueries

Definition and execution

A subquery is a SELECT statement embedded inside another SQL statement. The inner query is the subquery; the containing statement is the outer or main query. It is commonly used when the outer query depends on a value or set of values obtained from another query. Parentheses enclose the subquery.

```
SELECT empname, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee);
```

This returns employees whose salary exceeds the overall average. A non-correlated inner query can be evaluated independently; a correlated subquery refers to a row of the outer query and is evaluated using that row's values.

Types by result cardinality

Type	Result	Typical operators
Single-row	One value/row	=, ¡¿, ¿, ¡, ¿=, ¡=
Multiple-row	Several rows, usually one column	IN, ANY, ALL, EXISTS
Multiple-column	Several columns, possibly several rows	Tuple comparison, often IN

A single-row comparison must not return several rows. A multiple-row subquery needs a set operator such as IN, ANY, ALL or EXISTS. `ANY` is true if the comparison holds for at least one result; `ALL` is true if it holds for every result. `IN` tests membership. `NOT IN` can behave unexpectedly when the subquery returns NULL; `NOT EXISTS` is often safer for absence checks.

Placement and examples

Subqueries can appear in WHERE, HAVING, FROM and SELECT clauses, and can be used with data manipulation statements where allowed.

```
-- HAVING: compare grouped department averages
SELECT deptno, AVG(salary)
FROM employee
GROUP BY deptno
HAVING AVG(salary) >
  (SELECT AVG(salary) FROM employee WHERE deptno='D11');

-- IN: employees in a named department
SELECT empname FROM employee
WHERE deptno IN
  (SELECT deptno FROM department
   WHERE deptname='SOFTWARE SUPPORT');

-- EXISTS: employees assigned to a project
SELECT e.empno, e.empname
FROM employee e
WHERE EXISTS
  (SELECT 1 FROM emp_proj p WHERE p.empno=e.empno);

-- NOT EXISTS: employees with no project
SELECT e.empno, e.empname
FROM employee e
WHERE NOT EXISTS
  (SELECT 1 FROM emp_proj p WHERE p.empno=e.empno);
```

Correlated and uncorrelated subqueries

An **uncorrelated** subquery does not refer to the outer query. It can be evaluated first, and its result is passed to the outer query. A **correlated** subquery refers to a column from the current outer row, so it is logically evaluated for each candidate outer row.

```
-- Employees earning above their own department's average
SELECT e.empno, e.empname, e.deptno, e.salary
FROM employee e
WHERE e.salary >
  (SELECT AVG(e1.salary)
   FROM employee e1
   WHERE e1.deptno = e.deptno);
```

> **Remember. Read the correlation carefully:** the inner alias is compared to the outer alias, here `e1.deptno = e.deptno`. Without that link, the query would compare against one global average.

> **Exam Focus. Likely questions:** Explain correlated vs uncorrelated subqueries; distinguish single-row and multiple-row subqueries; explain IN, ANY, ALL and EXISTS with examples.

Views

Definition, purpose and creation

A view is a named virtual relation defined by a query. It presents selected data without requiring the user to work directly with every base-table column. A view can hide sensitive columns, simplify repeated queries, and present a purpose-specific representation. A view stores its defining query; it generally does not store an independent copy of the result.

```
CREATE VIEW emp_view AS
SELECT empname, deptno, gender, address
FROM employee;

SELECT empname FROM emp_view WHERE deptno='D01';
DROP VIEW emp_view;
```

Simple and complex views

Feature	Simple view	Complex view
Base tables	One	More than one
Grouping/aggregate	Usually absent	May include joins, groups, aggregates or expressions
DML through view	Often possible if restrictions are met	May be restricted or not updatable

View updates

A view is generally updatable when its query maps each view row clearly to a row in one base table. The supplied slides list conditions that restrict updates: DISTINCT, aggregate/summary functions, set operators, ORDER BY, multiple FROM tables, subqueries in WHERE, GROUP BY or HAVING, and calculated columns. For INSERT through a view, required NOT NULL base-table columns must be supplied. Exact update rules vary by DBMS and view definition.

> **Do Not Confuse.** A view is virtual, but an allowed UPDATE through it changes the underlying base table. Do not assume every view accepts INSERT, UPDATE or DELETE.

Transaction Control Language (TCL)

A transaction is a logical unit of database work. TCL commands control whether changes are saved or undone.

- `COMMIT;` permanently saves the current transaction's changes.
- `ROLLBACK;` undoes uncommitted changes since the last commit or rollback.
- `SAVEPOINT name;` marks a point within a transaction.
- `ROLLBACK TO name;` undoes work after that savepoint while retaining earlier work in the transaction.

```
SAVEPOINT before_raise;
UPDATE employee SET salary=salary*1.10;
ROLLBACK TO before_raise;  -- undo the raise
UPDATE employee SET salary=salary*1.20;
COMMIT;                    -- save this update
```

> **Remember.** After COMMIT, the committed changes cannot be undone with an ordinary ROLLBACK to an earlier savepoint. DDL transaction behavior depends on the DBMS; the supplied slides describe DDL as auto-committed in their Oracle context.

Data Control Language (DCL)

DCL manages privileges on database objects. A privilege may be granted to a user or role.

```
GRANT SELECT, INSERT ON employees TO user1;
GRANT SELECT ON employees TO user1 WITH GRANT OPTION;
REVOKE INSERT ON employees FROM user1;
REVOKE ALL PRIVILEGES ON employees FROM user1;
```

Privilege	Permitted action
SELECT	Read rows
INSERT	Add rows
UPDATE	Modify rows
DELETE	Remove rows
ALL PRIVILEGES	Grant all available object privileges

`WITH GRANT OPTION` lets the recipient grant the specified privilege to others. `GRANT` provides permissions; `REVOKE` removes previously granted permissions.

PL/SQL

What PL/SQL adds

PL/SQL is Oracle's procedural extension to SQL. It combines SQL statements with variables, decisions, loops, exception handling and cursors in a block. The supplied slides emphasize assignment with `:=`, output with `DBMS_OUTPUT.PUT_LINE`, loops, implicit cursor attributes and explicit cursors.

Block structure

```
DECLARE
  -- optional declarations: variables, cursors
BEGIN
  -- executable statements
EXCEPTION
  -- optional error handlers
END;
/
```

DECLARE and EXCEPTION are optional; BEGIN and END are required. The slash is commonly used in SQL*Plus to submit the completed block. Enable output in SQL*Plus with SET SERVEROUTPUT ON.

Variables, assignment and output

Declare a variable in DECLARE and assign values with :=. DBMS_OUTPUT.PUT_LINE displays a line in SQL*Plus when server output is enabled.

```
DECLARE
  student_name VARCHAR2(40);
BEGIN
  student_name := 'John Miller';
  DBMS_OUTPUT.PUT_LINE(student_name);
END;
/
```

Decision and repetition

```
IF condition THEN
  statements;
ELSIF another_condition THEN
  statements;
ELSE
  statements;
END IF;
```

PL/SQL supports basic LOOP, WHILE LOOP and FOR LOOP forms. A basic loop ends with END LOOP; use EXIT to leave it or EXIT WHEN condition for conditional termination. A WHILE loop repeats while its condition is true. A FOR loop iterates through a range.

```
LOOP
  statements;
  EXIT WHEN condition;
END LOOP;

WHILE condition LOOP
  statements;
END LOOP;

FOR i IN 1..5 LOOP
  DBMS_OUTPUT.PUT_LINE(i);
END LOOP;
```

Implicit cursor and SQL% attributes

Oracle creates an implicit cursor for DML and single-row SELECT INTO operations. It does not need declaration or explicit OPEN/FETCH/CLOSE. Cursor attributes report the most recent SQL statement's status: SQL%FOUND (affected/found at least one row), SQL%NOTFOUND, SQL%ROWCOUNT (number of rows affected/fetched), and SQL%ISOPEN (implicit cursor is generally closed after the statement).

```
UPDATE employee SET branchno = :branchno
WHERE empno = :empno;
IF SQL%FOUND THEN
  DBMS_OUTPUT.PUT_LINE('Employee transferred');
ELSE
  DBMS_OUTPUT.PUT_LINE('Employee not found');
END IF;
```

Explicit cursors

An explicit cursor handles a query that returns multiple rows. The programmer declares the query, opens the cursor, fetches rows into variables, checks for the end of data, and closes the cursor.

```
DECLARE
  CURSOR c_emp IS
    SELECT empcode, salary FROM employee WHERE job='analyst';
  v_emp employee.empcode%TYPE;
  v_salary employee.salary%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_emp, v_salary;
    EXIT WHEN c_emp%NOTFOUND;
    UPDATE employee SET salary = v_salary * 1.15
    WHERE empcode = v_emp;
  END LOOP;
  CLOSE c_emp;
END;
/
```

A cursor FOR loop can handle OPEN, FETCH and CLOSE automatically. For an explicit cursor, common attributes are `c_emp%FOUND`, `c_emp%NOTFOUND`, `c_emp%ROWCOUNT` and `c_emp%ISOPEN`.

> **Do Not Confuse.** An explicit cursor fetch must run inside a loop or equivalent repeated logic. Exit when `cursor%NOTFOUND` immediately after FETCH to avoid processing an invalid final row. Use an implicit cursor for a single-row DML status check; use an explicit cursor to process query rows.

Important Exam Questions

Definitions / short answers

1. Define a join and distinguish its condition from its type.
2. Define a subquery. What are its inner and outer queries?
3. Define a view and state two uses.
4. What do COMMIT, ROLLBACK and SAVEPOINT do?
5. What is the purpose of GRANT and REVOKE?
6. What is an implicit cursor? Name SQL cursor attributes.
7. What is an explicit cursor? List its processing steps.

Five-mark questions

1. Compare inner join, left outer join, right outer join and full outer join with examples.
2. Explain single-row, multiple-row and multiple-column subqueries.
3. Compare correlated and uncorrelated subqueries with SQL examples.
4. Explain simple and complex views and conditions affecting view updates.
5. Explain TCL commands with a savepoint example.
6. Explain PL/SQL block structure, variables, output and control statements.
7. Explain implicit and explicit cursors and relevant attributes.

Long-answer / query questions

1. Write SQL to list employees with salary above their department's average.
2. Write SQL using EXISTS and NOT EXISTS to list employees assigned and not assigned to any project.

3. Explain join types with syntax and result behavior for unmatched rows.
4. Explain GRANT, REVOKE, privileges and WITH GRANT OPTION with syntax.
5. Write a PL/SQL block using an explicit cursor to raise analyst salaries by 15% and report whether rows were found.

Last-Minute Revision

JOINS
Inner keeps matches. Left/right/full outer preserve rows from one/both sides.

SUBQUERIES
Single row: comparison operators. Multiple row: IN, ANY, ALL, EXISTS. Correlated: refers to outer row.

VIEWS
Named virtual relation based on a query. Updates may pass through to base table only when allowed.

TCL
COMMIT saves. ROLLBACK undoes uncommitted changes. SAVEPOINT marks a partial rollback point.

DCL
GRANT gives privileges. REVOKE removes privileges.

PL/SQL
DECLARE (optional), BEGIN (required), EXCEPTION (optional), END. Explicit cursor: declare, open, fetch, exit on NOTFOUND, close.

Source Scope

Prepared from the supplied Module 4 slide decks: SQL joins, SQL subqueries, views/TCL/DCL, and PL/SQL. Oracle-style examples and terminology follow those slides; SQL details can vary by database product.